

Experiment-08: Depth-First Search (DFS) using Python

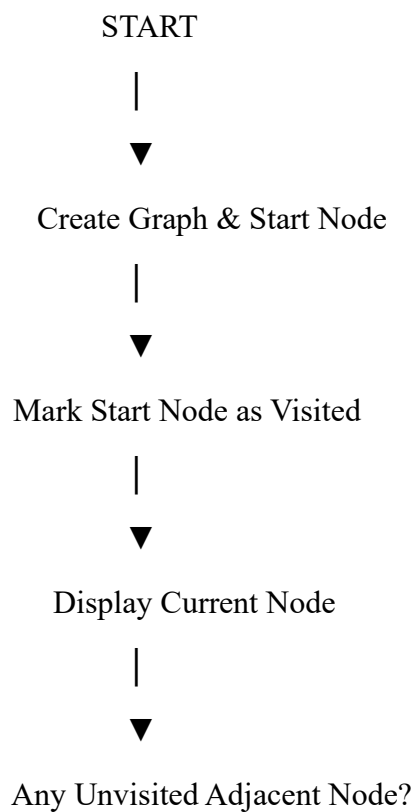
Aim

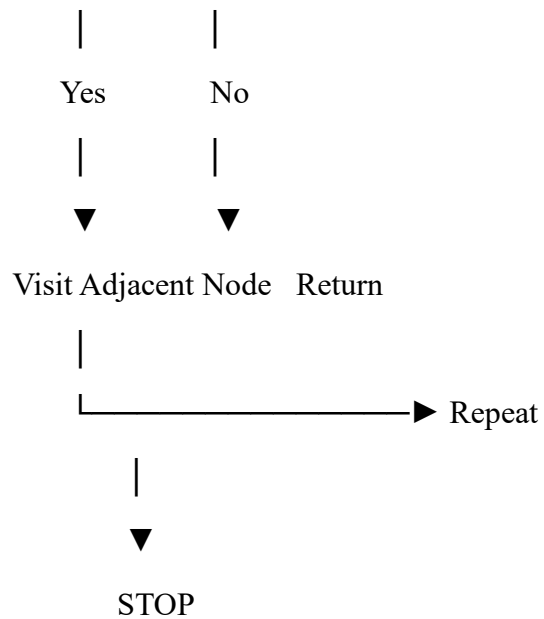
To develop a Python program to implement the **Depth-First Search (DFS)** algorithm for traversing a graph and visiting all reachable nodes using the **Depth-First** approach.

Algorithm

1. Create the graph using an adjacency list.
 2. Choose the starting node.
 3. Create an empty set to store visited nodes.
 4. Mark the starting node as visited and display it.
 5. Visit each unvisited adjacent node recursively.
 6. Repeat the process until all reachable nodes are visited.
 7. Stop.
-

Flowchart





Python Code

DFS using Recursion

```
def dfs(graph, node, visited):
```

```
    if node not in visited:
```

```
        print(node, end=" ")
```

```
        visited.add(node)
```

```
    for neighbor in graph[node]:
```

```
        dfs(graph, neighbor, visited)
```

Graph Representation

```
graph = {
```

```
    'A': ['B', 'C'],
```

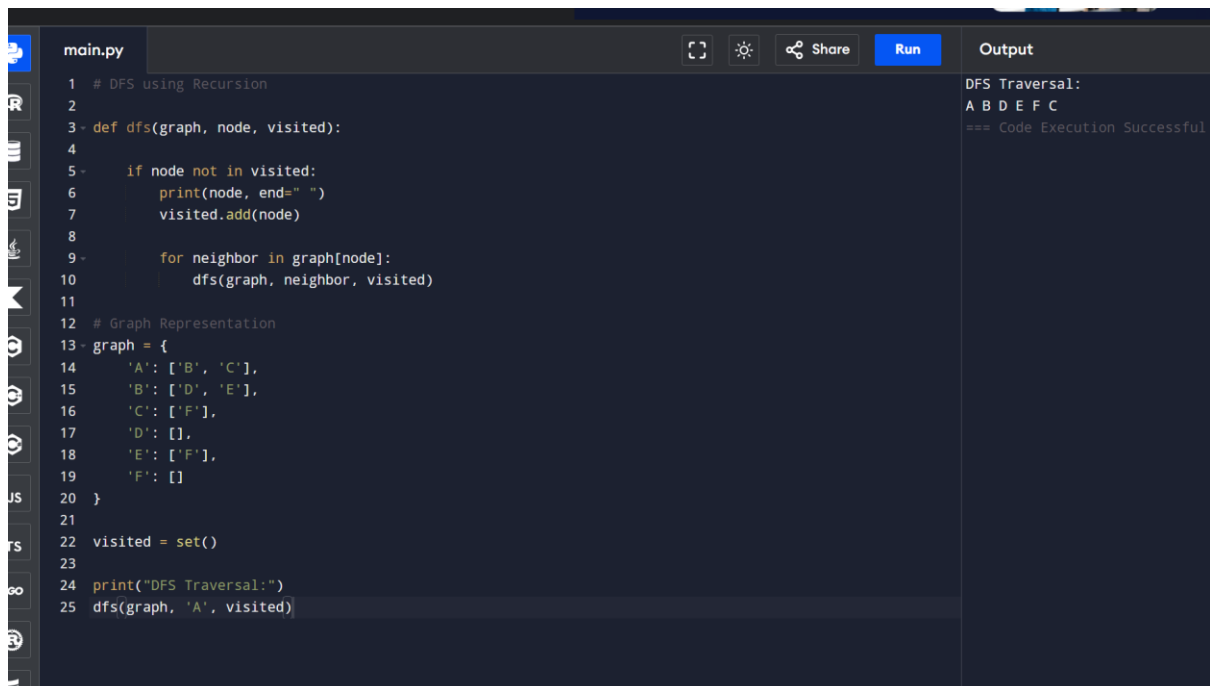
```
    'B': ['D', 'E'],
```

```
    'C': ['F'],
```

```
    'D': [],
```

```
'E': ['F'],  
'F': []  
}  
  
visited = set()  
  
print("DFS Traversal:")  
dfs(graph, 'A', visited)
```

Output



The screenshot shows a code editor with a file named 'main.py'. The code implements a Depth-First Search (DFS) algorithm using recursion. It defines a function 'dfs' that takes a graph, a current node, and a set of visited nodes as arguments. The function prints the current node and its neighbors, then recursively visits each neighbor. The graph is represented as a dictionary with nodes 'A' through 'F' and their respective neighbors. The code then initializes a 'visited' set, prints 'DFS Traversal:', and calls the 'dfs' function starting from node 'A'.

```
main.py  
1 # DFS using Recursion  
2  
3 def dfs(graph, node, visited):  
4  
5     if node not in visited:  
6         print(node, end=" ")  
7         visited.add(node)  
8  
9         for neighbor in graph[node]:  
10             dfs(graph, neighbor, visited)  
11  
12 # Graph Representation  
13 graph = {  
14     'A': ['B', 'C'],  
15     'B': ['D', 'E'],  
16     'C': ['F'],  
17     'D': [],  
18     'E': ['F'],  
19     'F': []  
20 }  
21  
22 visited = set()  
23  
24 print("DFS Traversal:")  
25 dfs(graph, 'A', visited)
```

The output panel on the right shows the result of the DFS traversal: 'DFS Traversal: A B D E F C' followed by '=== Code Execution Successful'.

Result

The **Depth-First Search (DFS)** algorithm was successfully implemented in Python using recursion. The program traversed the graph by visiting each node as deeply as possible before backtracking. The traversal order obtained was:

A → B → D → E → F → C